Bitácora S08: Mexi Banco

Que la transferencia no deje dinero en el aire

Sistemas Distribuidos (Semestre 2027-1, FI-UNAM)

Victor Emiliano Calderón Gutiérrez

No. de cuenta 423097331

17 de octubre de 2026

Entorno

Hora: 04:18

Para este laboratorio partí de la rama `03-gateway-discovery-balanceo` del repositorio https://github.com/OscarRuiz21/mexi-banco, que es la solución publicada del lab S07, y la cloné dentro de mi carpeta de entregas como `s08/mexi-banco`. Igual que la vez pasada, todo lo corrí en Windows 11 con Docker y Git Bash, y desde el clon desactivé `core.autocrlf` para que `mvnw` no se volviera a romper con los finales de línea de Windows.

Antes de levantar nada tuve que hacer un cambio en `docker-compose.yml`: la rama base le pone al proyecto el nombre `mexi-banco-v06`, que es justo el mismo que tenía del lab S07 (con sus contenedores, su red y su volumen con los datos de ese lab), por lo que si lo levantaba así iba a reusar esa base de datos. Para empezar limpio sin borrar lo del otro lab, le cambié el nombre a `mexi-banco-s08` (y a la red `mexibanco-s08`). También me tocó un error que no tenía que ver con el código: en la primera construcción la descarga de Maven dentro de la imagen de `spei` salió corrupta (`unzip: crc error`), por lo que volví a lanzar `docker compose up -build -d`. En ese segundo intento cinco imágenes salieron bien, pero las de `notificacion` y `spei` se quedaron colgadas en una descarga de Maven (lo intenté dos veces, y la segunda pasaron más de veinte minutos sin avanzar), así que las dejé fuera y levanté el resto con `docker compose up -d -no-build`. Para este lab no hacían falta, ya que `spei` no participa en las transferencias internas y los avisos de `notificacion` van al final de la saga, y de hecho me sirvió para comprobar que una transferencia se completa aunque el aviso falle (lo explico en la sección 4). Por último, como el entorno del S07 ocupaba los mismos puertos (8080 y 8761), lo detuve con `docker compose stop`, que no borra nada.

1. El problema: dinero en el aire

Hora: 04:47

Tal como lo pide la guía, antes de tocar el código reproduje el problema con la versión original de `transferencia`. Para esto di de alta dos cuentas por el gateway, la de Ana (`0021800000000000801`) con 1000 y la de Beto (`0021800000000000802`) con 500, y consulté sus saldos.

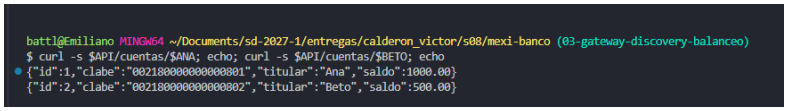


Figura 1: Saldos antes de la transferencia: Ana 1000 y Beto 500.

Después volví a levantar `transferencia` con una pausa de 20 segundos entre el cargo y el abono (`PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia`), y en una sola línea lancé una transferencia de 200 de Ana a Beto en segundo plano y, cuatro segundos después (es decir, ya dentro de la pausa), apagué las tres copias de `cuenta`:

```
date +%T; curl -s -w '\n[HTTP %{http_code} en %{time_total} s]\n' -X POST $API/transferencias \
  -H 'Content-Type: application/json' \
  -d "{\"claveOrigen\":\"$ANA\",\"claveDestino\":\"$BETO\",\"monto\":200}" & \
  sleep 4; date +%T; docker compose stop cuenta; wait; date +%T
```

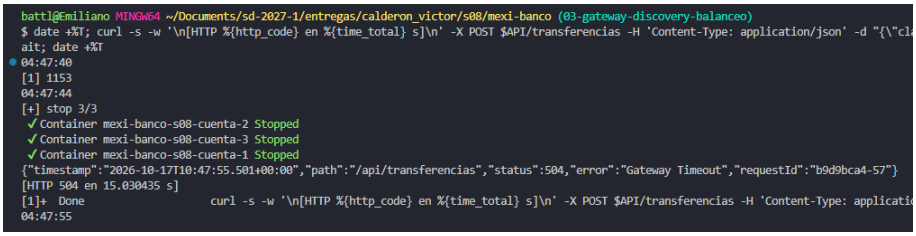


Figura 2: La transferencia sale a las 04:47:40, `cuenta` se apaga a las 04:47:44 y el cliente recibe un 504 a los 15 segundos.

Lo primero que me llamó la atención es que el cliente ni siquiera se enteró de lo que pasó: a los 15.03 segundos recibió un `504 Gateway Timeout`, ya que el gateway tiene un `response-timeout` de 15 segundos y la pausa era de 20, por lo que se cansó de esperar antes de que `transferencia` terminara. Mientras tanto, `transferencia` siguió trabajando por detrás, y en sus logs se ve que justo a los 20 segundos intentó el abono y ya no encontró a nadie (el balanceador no tenía ninguna copia de `cuenta` en su lista), lo que terminó en un 500 que ya no le llegó a nadie:

```
04:48:00.636  WARN  RoundRobinLoadBalancer : No servers available for service: cuenta
04:48:00.641 ERROR  dispatcherServlet      : Servlet.service() ... threw exception [Request
             processing failed: java.lang.IllegalStateException: No instances available for cuenta]
```

Ya con `cuenta` encendida otra vez (`docker compose start cuenta`) volví a consultar los saldos y el estado de cuenta de Ana:

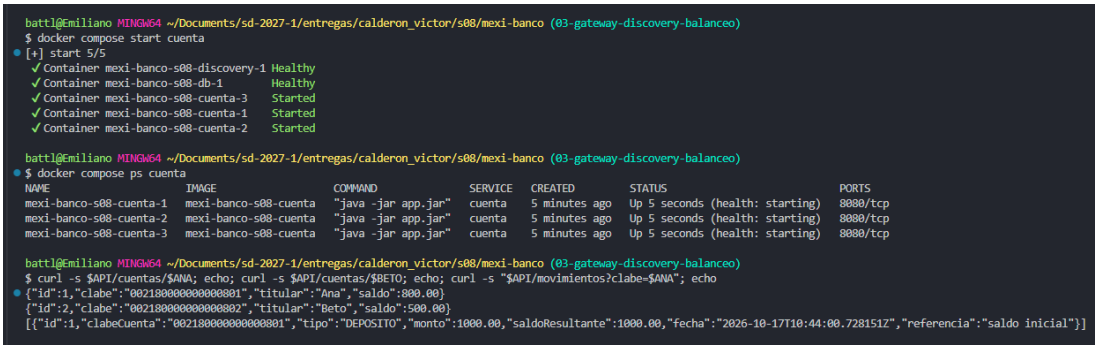


Figura 3: Con `cuenta` de vuelta: Ana 800, Beto 500, y en el estado de cuenta de Ana solo está su depósito inicial.

Es decir, Ana pasó de 1000 a 800 y Beto se quedó en 500: los 200 salieron de una cuenta y no llegaron a la otra. Y algo que lo hace peor es que en el estado de cuenta de Ana no hay ningún movimiento que lo explique (solo aparece el "saldo inicial"), porque en la versión original los asientos se registran hasta después del abono, que nunca pasó.

¿Por qué pasó? Porque el cargo y el abono son dos peticiones distintas a `cuenta`, cada una confirmada en su propia transacción local, y `transferir()` no tiene nada que abarque a las dos: cuando apagué `cuenta` el cargo ya estaba confirmado, y cuando el abono falló nadie lo deshizo.

2. Mi saga

Hora: 04:35

Tal como lo vimos en la clase, una saga es una secuencia de transacciones locales en la que cada paso tiene una *compensación* que deshace su efecto de negocio, y en mi caso es una saga *orquestada*: `TransferenciaService` es el que decide qué paso sigue y qué hacer si uno falla, y en lugar de un `@Transactional` que ya no puede abarcar tres procesos y cuatro bases, lo que tiene es un plan para regresar. Los pasos y la compensación de cada uno quedaron así:

Paso	**Servicio**	**Compensación / qué pasa si falla**
1. Consultar origen y destino	`cuenta`	No cambia nada, por lo que no necesita compensación: si falla, la transferencia queda `RECHAZADA`.
2. Cargo a origen	`cuenta`	Su compensación es **devolverle el monto a origen** (un abono a origen) y dejar el asiento de la devolución. Si el cargo mismo falla, `cuenta` no aplicó nada (es una transacción local) y queda `RECHAZADA`.
3. Asiento del cargo	`movimiento`	Lo cubre la devolución: el estado de cuenta muestra el cargo y luego la devolución.
4. Abono a destino	`cuenta`	Es el paso que decide (el "pivote"): si falla, se compensa el paso 2; si sale bien, ya no se regresa.
5. Asiento del abono	`movimiento`	Ya no se compensa, porque el dinero ya llegó: si falla, la transferencia se queda `ABONADA` y se anota lo que falta.
6. Avisos	`notificacion`	No se pueden deshacer (un aviso enviado no se "desenvía"), por eso van hasta el final y si fallan solo se registran en el log.

El estado de cada transferencia se guarda en su tabla y cambia en cada paso, es decir, si el servicio se cae a la mitad, la base dice exactamente en qué paso se quedó. El diagrama de estados es el siguiente (los de doble borde son finales):

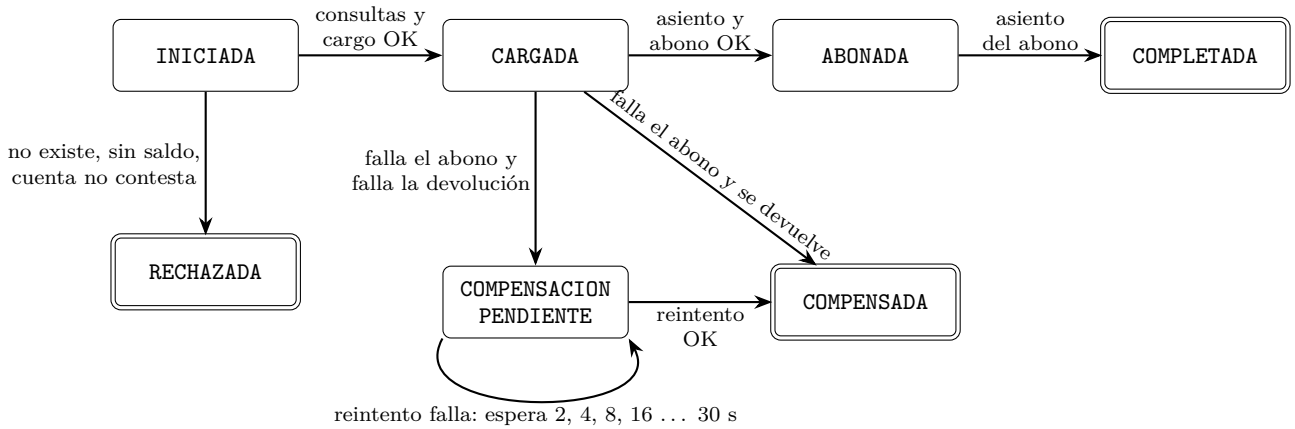


Figura 4: Diagrama de estados de una transferencia.

Algo que tuve que decidir es cómo dejar escrita la devolución en el estado de cuenta. Lo ideal sería un tipo de movimiento `DEVOLUCION`, pero el enum de `movimiento` solo tiene `DEPOSITO`, `TRANSFERENCIA_ENVIADA`, `TRANSFERENCIA_RECIBIDA` y `SPEI_ENVIADO`, y agregar uno obligaría a cambiar y redesplegar `movimiento` al mismo tiempo que `transferencia` (que es justo el precio de no compartir código que menciona el mismo proyecto). Por lo que usé `TRANSFERENCIA_RECIBIDA` con la referencia "devolucion de la transferencia N", así el dinero entra a origen y el asiento dice de dónde viene.

3. El código clave

Hora: 04:40

Todos los cambios de código están en `transferencia`, y en el código los marqué con comentarios "Lab S08". Aparte, en `docker-compose.yml` cambié el nombre del proyecto y de la red (lo de la sección de entorno), agregué la variable `FALLA_ABONO_CLABE` y le puse a `transferencia` la zona horaria `TZ=America/Mexico_City`, para que la hora de sus logs coincidiera con la de esta bitácora. Lo más importante del código es lo siguiente.

3.1 · El estado en la tabla

A la entidad `Transferencia` le agregué el estado, un detalle en palabras, el motivo de la falla, cuántas veces se ha intentado devolver el cargo, cuándo toca el siguiente intento y un `@Version`. La fila se crea `INICIADA` antes de mover un solo peso, y cada cambio de estado pasa por un método de la entidad, para que no se pueda poner un estado sin su detalle:

```
@Enumerated(EnumType.STRING) @Column(length = 30)
private EstadoTransferencia estado;
@Column(length = 500) private String motivo;   // lo que hizo fallar la saga
@Column(length = 500) private String detalle;  // el último paso, en palabras
private Instant actualizada;
@ColumnDefault("0") private int intentosCompensacion;
private Instant proximoIntento;
@Version private Long version;   // dos procesos guardando la misma fila: el segundo truena
```

3.2 · El orquestador

`transferir()` quedó dividido en los pasos de la tabla de arriba, y cada bloque sabe qué hacer si algo falla. Lo que cambia entre un bloque y otro es si el dinero ya se movió o no: antes del cargo, cualquier error deja la transferencia `RECHAZADA`, mientras que después del cargo cualquier error (el asiento, la falla provocada o el abono) manda a compensar:

```
Transferencia t = transferencias.save(new Transferencia(origen, destino, monto, reloj.instant()));
try {                                          // pasos 1 y 2: si falla, no se movió nada
  cuentas.consultar(claveOrigen);
  cuentas.consultar(claveDestino);
  origen = cuentas.cargar(claveOrigen, monto);
} catch (RuntimeException e) {
  t.rechazada("No se movio dinero: " + e.getMessage(), reloj.instant());
  t = transferencias.save(t);
  throw new TransferenciaFallidaException(t, codigoHttp(e), t.getDetalle());
}
t.cargada(origen.saldo(), reloj.instant());
t = transferencias.save(t);                    // CARGADA: desde aquí el dinero ya salió
try {                                          // paso 3: si falla, se compensa
  cargo = movimientos.registrar(origen.clabe(), TRANSFERENCIA_ENVIADA, monto.negate(), ...);
  pausaDeDemo();
  fallaProvocada(claveDestino);
  destino = cuentas.abonar(claveDestino, monto);
} catch (RuntimeException e) {
  t.porCompensar("El abono a " + claveDestino + " fallo: " + e.getMessage());
  t = compensar(t);
  throw new TransferenciaFallidaException(t, 503, t.getMotivo() + ". " + t.getDetalle());
}
t.abonada(destino.saldo(), reloj.instant());   // ABONADA, luego el asiento y COMPLETADA
```

Cuando la saga no se completa, la respuesta ya no es un 500 sin más: `ErroresDeLaSaga` convierte la `TransferenciaFallidaException` en un Problem Details que, además del detalle (qué falló y qué se hizo con el dinero), trae el `id` de la transferencia y su `estado`, para que el cliente sepa si su dinero ya regresó o sigue pendiente.

3.3 · La compensación

`compensar()` es la que devuelve el cargo, y la llaman tanto la saga (el primer intento) como el reintento programado (los siguientes). Como se puede llamar más de una vez para la misma transferencia, la escribí cuidando que repetirla no devuelva el dinero dos veces:

```
Transferencia compensar(Transferencia t) {
  if (t.getEstado() != CARGADA && t.getEstado() != COMPENSACION_PENDIENTE) {
    return t;            // ya se devolvió (o nunca hubo cargo): repetirla no hace nada
  }
  int intento = t.nuevoIntentoDeCompensacion(reloj.instant().plus(APARTADA_POR));
  t = transferencias.save(t);   // "aparta" la fila; si otro proceso ya la apartó, @Version truena aquí
  CuentaRemota origen;
  try {
    origen = cuentas.abonar(t.getClaveOrigen(), t.getMonto());
  } catch (RuntimeException e) {
    Duration espera = esperaAntesDelIntento(intento + 1);      // 2, 4, 8, 16... hasta 30 s
    t.compensacionPendiente("La devolucion fallo en el intento " + intento + ...,
        reloj.instant(), reloj.instant().plus(espera));
    return transferencias.save(t);
  }
  t.compensada("Se devolvieron " + t.getMonto() + " a " + t.getClaveOrigen() + ...);
  t = transferencias.save(t);   // lo primero después de que cuenta confirma: COMPENSADA
```

```
  movimientos.registrar(origen.clabe(), TRANSFERENCIA_RECIBIDA, t.getMonto(), origen.saldo(),
      "devolucion de la transferencia " + t.getId() + ...);   // (si falla, se anota y ya)
  return t;
}
```

3.4 · La falla provocada

Para probar la compensación con `cuenta` arriba usé una variable de entorno, `FALLA_ABONO_CLABE`: si la CLABE destino de una transferencia está ahí (acepta varias separadas por comas), el abono falla a propósito antes de llamar a `cuenta`. Algo importante es que esta falla no pasa por el circuit breaker, ya que no es una falla real de `cuenta` y no quería que mis pruebas de compensación abrieran el circuito. En `docker-compose.yml` quedó con valor vacío por omisión, por lo que se activa solo al levantar `transferencia`:

```
FALLA_ABONO_CLABE=002180000000000812 docker compose up -d transferencia
```

3.5 · El reintento programado

Para que una transferencia no se quede en `COMPENSACION_PENDIENTE` para siempre agregué `ReintentoDeCompensaciones`, un `@Scheduled` que cada segundo busca en la base las devoluciones pendientes a las que ya les toca su siguiente intento y llama otra vez a `compensar()`. La espera entre intentos va creciendo (2, 4, 8, 16 y luego 30 segundos como máximo), ya que si `cuenta` está caída no tiene caso insistirle cada segundo, y como todo sale de la base, también retoma las pendientes que quedaron de antes de un reinicio de `transferencia`.

```
@Scheduled(fixedDelayString = "${mexibanco.saga.revisar-cada-ms:1000}")
void reintentarPendientes() {
  for (Transferencia t : transferencias.findByEstadoAndProximoIntentoLessThanEqualOrderByIdAsc(
      COMPENSACION_PENDIENTE, Instant.now())) {
    try {
      saga.compensar(t);
    } catch (OptimisticLockingFailureException otroProcesoLaTomo) { ... }
  }
}
```

3.6 · El circuit breaker, hecho a mano

Decidí hacerlo a mano en lugar de usar Resilience4j. Es una clase `CircuitBreaker` con los tres estados de la clase, un contador de fallas seguidas y un tiempo de espera, y todas las llamadas de `CuentaCliente` pasan por ella. Con la configuración que dejé, tres fallas seguidas lo abren, se queda abierto 15 segundos sin llamar a `cuenta`, y después deja pasar una sola llamada de prueba (semiabierto): si sale bien se cierra, y si falla se vuelve a abrir con la espera desde cero.

```
public <T> T llamar(Supplier<T> llamada) {
  antesDeLlamar();            // ABIERTO: lanza CircuitoAbiertoException sin tocar la red
  try {
    T resultado = llamada.get();
    alSalirBien();            // SEMIABIERTO -> CERRADO, y reinicia la cuenta de fallas
    return resultado;
  } catch (RuntimeException e) {
    if (esFalla.test(e)) alFallar(e); else alSalirBien();
    throw e;
  }
}
```

```
private synchronized void antesDeLlamar() {
  if (estado == ABIERTO) {
    Duration falta = Duration.between(reloj.instant(), abiertoDesde.plus(espera));
    if (falta.isPositive()) throw new CircuitoAbiertoException(nombre, ...);
    cambiarA(SEMIABIERTO, "pasaron " + espera.toSeconds() + " s: ...");
  }
  if (estado == SEMIABIERTO) {
    if (pruebaEnCurso) throw new CircuitoAbiertoException(nombre, "ya hay una llamada de prueba en
    curso");
    pruebaEnCurso = true;
  }
}
```

Algo que me pareció importante es qué cuenta como falla. Solo cuentan las que dicen que `cuenta` está mal (que no contestó, que se pasó del tiempo, que respondió 5xx o que el directorio no tiene instancias), mientras que un 404 o un 422 por saldo insuficiente no abren el circuito, porque ahí `cuenta` sí contestó bien, solo que la respuesta fue que no. Cada cambio de estado se escribe en el log con su motivo y se guarda con la hora, y para verlo agregué un endpoint propio en `/actuator/circuitbreakers` (el mismo nombre que usa Resilience4j).

3.7 · Timeout explícito en las llamadas a cuenta

`ClientesHttp` ya traía un tiempo máximo general (2 s para conectar y 5 s para responder), pero para `cuenta` le puse uno propio y más corto en `CuentaCliente`: 1 segundo para conectar y 2 segundos para responder (configurables con `CUENTA_TIMEOUT_CONEXION_MS` y `CUENTA_TIMEOUT_RESPUESTA_MS`). Además, lo que no es una respuesta de `cuenta` (que no conteste, que se pase del tiempo o que el balanceador no encuentre instancias) se convierte en un 503 que dice qué pasó, en lugar del 500 que salía antes cuando el directorio se quedaba sin copias.

3.8 · Pruebas

Por último actualicé las pruebas que ya traía el servicio (la cuarta, que antes documentaba que nadie deshacía el cargo, ahora comprueba la compensación) y agregué las de la devolución pendiente, la falla provocada, la espera creciente y los tres estados del circuito (con un reloj que avanza a mano). Corrí las 16 dentro de un contenedor con el JDK 21 y pasaron todas (resumo la salida de Maven):

```
Tests run: 1, Failures: 0, Errors: 0 -- in mx.mexibanco.clientes.BalanceoPorNombreTest
Tests run: 3, Failures: 0, Errors: 0 -- in mx.mexibanco.clientes.ClientesHttpTest
Tests run: 5, Failures: 0, Errors: 0 -- in mx.mexibanco.resiliencia.CircuitBreakerTest
Tests run: 7, Failures: 0, Errors: 0 -- in mx.mexibanco.transferencia.TransferenciaServiceTest
Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

4. La compensación funcionando

Hora: 04:51

Para probar la versión nueva copié mi `transferencia` al clon y la levanté sola, ya con la falla provocada apuntando a la CLABE de Diego:

```
FALLA_ABONO_CLABE=002180000000000812 docker compose up --build -d transferencia
```

Antes de provocar la falla hice una transferencia normal entre dos cuentas de prueba, para comprobar que el camino feliz siguiera funcionando, y en los logs se ve cómo va pasando por cada estado en menos de un cuarto de segundo. Algo que también salió aquí es que, como `notificacion` no estaba levantado, los dos avisos fallaron, pero la transferencia quedó `COMPLETADA` de todas formas, que es justo lo que buscaba al dejar los avisos hasta el final:

```
04:51:27.134  transferencia 1: INICIADA (002180000000000901 -> 002180000000000902, 100)
04:51:27.298  transferencia 1: CARGADA (origen queda en 900.00)
04:51:27.334  transferencia 1: ABONADA (destino queda en 600.00)
04:51:27.342  transferencia 1: COMPLETADA
04:51:27.351  transferencia 1: no se pudo mandar un aviso (No instances available for notificacion);
              el dinero ya quedo bien
```

Después di de alta a Carla (`002180000000000811`) con 1000 y a Diego (`002180000000000812`) con 500, y le mandé 200 de Carla a Diego. Como la CLABE de Diego es la de `FALLA_ABONO_CLABE`, el abono falló a propósito después del cargo, y esta vez la respuesta ya no fue un 500 sin más: es un 503 que dice qué falló, que se le devolvieron los 200 a Carla, el `id` de la transferencia y que quedó `COMPENSADA`.

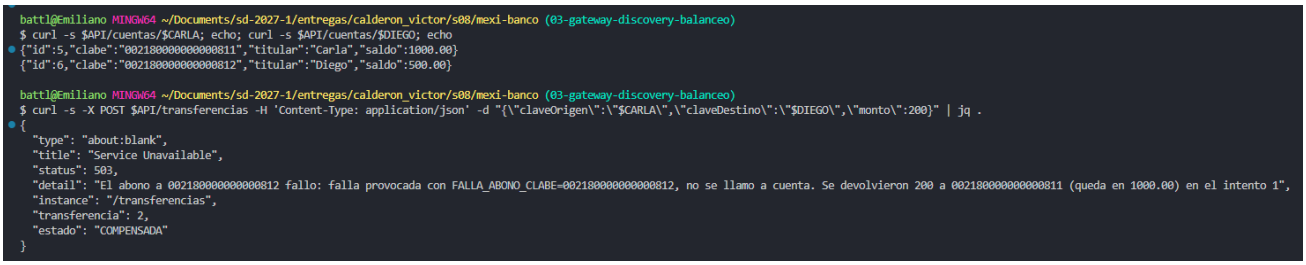


Figura 5: Saldos antes (Carla 1000, Diego 500) y la respuesta de la transferencia que falla a propósito.

Al volver a consultar, los saldos regresaron a 1000 y 500, y en el estado de cuenta de Carla se ve toda la historia: primero el cargo de -200 (saldo 800) y luego la devolución de $+200$ (saldo 1000) con la referencia “devolucion de la transferencia 2”. Es decir, el dinero sí salió y sí regresó, y las dos cosas quedaron escritas, en lugar de desaparecer como en la Parte 0. Consultando la transferencia por su `id` también se ve el motivo, el detalle y que se resolvió en el primer intento:

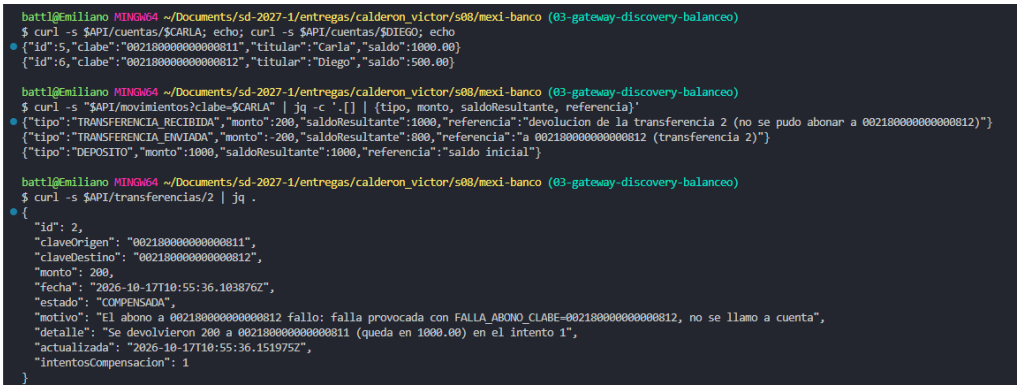


Figura 6: Los saldos regresan, el estado de cuenta de Carla tiene el cargo y la devolución, y `GET /api/transferencias/2`.

Por último, en la base de `transferencia` quedan las dos filas: la 1 `COMPLETADA` y la 2 `COMPENSADA`, con un intento de compensación y el motivo y el detalle en palabras.

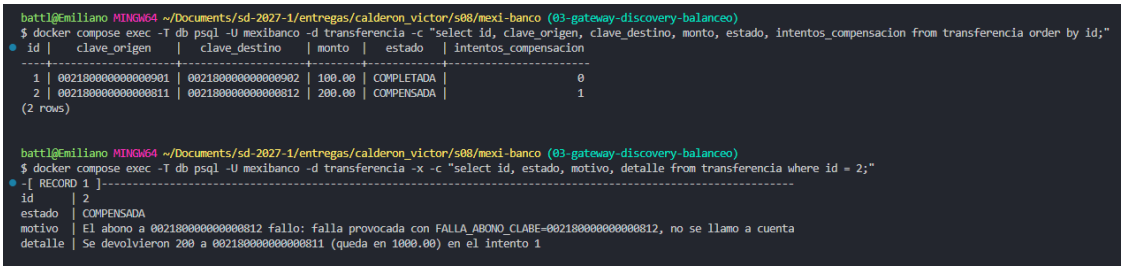


Figura 7: El estado final en la base de `transferencia`.

En los logs se ve que toda la saga, compensación incluida, tardó 43 milisegundos:

```
04:55:36.111  transferencia 2: INICIADA (002180000000000811 -> 002180000000000812, 200)
04:55:36.136  transferencia 2: CARGADA (origen queda en 800.00)
04:55:36.141  transferencia 2: fallo despues del cargo (falla provocada con
              FALLA_ABONO_CLABE=002180000000000812, no se llamo a cuenta); se compensa
04:55:36.154  transferencia 2: COMPENSADA en el intento 1 (origen regresa a 1000.00)
```

5. Cuando la compensación también falla

Hora: 05:04

Para este caso repetí la Parte 0, pero ya con mi versión: volví a levantar `transferencia` con la pausa de 20 segundos, le mandé 200 de Elena (`00218000000000821`, con 1000) a Fer (`00218000000000822`, con 500) y apagué `cuenta` durante la pausa (la captura de ese momento la uso en la sección 7). El cliente otra vez recibió un 504 a los 15 segundos, pero esta vez eso ya no importa tanto, porque la base sí sabe en qué va la transferencia. Lo que pasó después se ve en los logs:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s08/mexi-banco (03-gateway-discovery-balanceo)
$ docker compose logs --no-log-prefix transferencia --since 3m | grep -E 'transferencia [0-9]+:|circuito cuenta' | sed -E 's/^.{11}(.{8}).*(Service|Breaker|Compensaciones) +: /\1  /'
05:04:10  transferencia 11: INICIADA (00218000000000821 -> 00218000000000822, 200)
05:04:10  transferencia 11: CARGADA (origen queda en 800.00)
05:04:30  circuito cuenta: falla 1 de 3 (el directorio no tiene ninguna instancia de cuenta)
05:04:30  transferencia 11: fallo despues del cargo (el directorio no tiene ninguna instancia de cuenta); se compensa
05:04:30  circuito cuenta: falla 2 de 3 (el directorio no tiene ninguna instancia de cuenta)
05:04:30  transferencia 11: COMPENSACION_PENDIENTE, el intento 1 de devolver fallo (el directorio no tiene ninguna instancia de cuenta); siguiente en 2 s
05:04:33  transferencia 11: reintento programado de la devolucion (intento 2)
05:04:33  circuito cuenta: falla 3 de 3 (el directorio no tiene ninguna instancia de cuenta)
05:04:33  circuito cuenta: CERRADO -> ABIERTO (3 fallas seguidas; no se llama a cuenta en 15 s)
05:04:33  transferencia 11: COMPENSACION_PENDIENTE, el intento 2 de devolver fallo (el directorio no tiene ninguna instancia de cuenta); siguiente en 4 s
05:04:37  transferencia 11: reintento programado de la devolucion (intento 3)
05:04:37  transferencia 11: COMPENSACION_PENDIENTE, el intento 3 de devolver fallo (No se llamo a cuenta: el circuito esta ABIERTO (se vuelve a probar en 11 s; ultima falla: el directorio no tiene ninguna instancia de cuenta)); siguiente en 8 s
05:04:45  transferencia 11: reintento programado de la devolucion (intento 4)
05:04:45  transferencia 11: COMPENSACION_PENDIENTE, el intento 4 de devolver fallo (No se llamo a cuenta: el circuito esta ABIERTO (se vuelve a probar en 3 s; ultima falla: el directorio no tiene ninguna instancia de cuenta)); siguiente en 16 s
```

Figura 8: Los logs de `transferencia`: el abono falla, la devolución también, y cada reintento espera más que el anterior.

Como la captura queda algo chica, aquí van las mismas líneas (recorté con "(...)" los motivos que se repiten):

```
05:04:10  transferencia 11: INICIADA (00218000000000821 -> 00218000000000822, 200)
05:04:10  transferencia 11: CARGADA (origen queda en 800.00)
05:04:30  transferencia 11: fallo despues del cargo (el directorio no tiene ninguna instancia
          de cuenta); se compensa
05:04:30  transferencia 11: COMPENSACION_PENDIENTE, el intento 1 de devolver fallo (...); siguiente en 2 s
05:04:33  circuito cuenta: CERRADO -> ABIERTO (3 fallas seguidas; no se llama a cuenta en 15 s)
05:04:33  transferencia 11: COMPENSACION_PENDIENTE, el intento 2 de devolver fallo (...); siguiente en 4 s
05:04:37  transferencia 11: COMPENSACION_PENDIENTE, el intento 3 de devolver fallo (No se llamo a
          cuenta: el circuito esta ABIERTO (...)); siguiente en 8 s
05:04:45  transferencia 11: COMPENSACION_PENDIENTE, el intento 4 de devolver fallo (No se llamo a
          cuenta: el circuito esta ABIERTO (...)); siguiente en 16 s
05:05:01  circuito cuenta: ABIERTO -> SEMIABIERTO -> ABIERTO (la llamada de prueba fallo)
05:05:01  transferencia 11: COMPENSACION_PENDIENTE, el intento 5 de devolver fallo (...); siguiente en 30 s
```

Es decir, a los 20 segundos el abono falló (el directorio ya no tenía copias de `cuenta`), la saga intentó devolverle el dinero a Elena en ese mismo momento y también falló, por lo que la transferencia quedó en `COMPENSACION_PENDIENTE`. A partir de ahí el reintento programado la volvió a intentar a los 2, 4, 8, 16 y luego cada 30 segundos. También se ve cómo trabajan juntos la saga y el circuit breaker: el circuito contó el abono y los dos primeros intentos de devolver, a la tercera falla se abrió, y por eso los intentos 3 y 4 ya ni siquiera llamaron a `cuenta`. En la base, la fila decía exactamente en qué iba (seis intentos, la hora del siguiente y el motivo):

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s08/mexi-banco (03-gateway-discovery-balanceo)
$ docker compose exec -T db psql -U mexibanco -d transferencia -x -c "select id, estado, intentos_compensacion, proximo_intento, motivo, detalle from transferencia where clave_origen = '$ELENA';"
-[ RECORD 1 ]---------+-----------------------------------------------------------------------------------------------------------------------
id                    | 11
estado                | COMPENSACION_PENDIENTE
intentos_compensacion | 6
proximo_intento       | 2026-10-17 11:06:01.513558+00
motivo                | El abono a 00218000000000822 fallo: el directorio no tiene ninguna instancia de cuenta
detalle               | La devolucion fallo en el intento 6 (el directorio no tiene ninguna instancia de cuenta); el siguiente intento va en 30 s
```

Figura 9: La transferencia 11 en `COMPENSACION_PENDIENTE` (la hora de `proximo_intento` está en UTC: 05:06:01 en hora de México).

Ya que encendí `cuenta` otra vez, el reintento de las 05:06:31 fue también la llamada de prueba del circuito: salió bien, el circuito se cerró y la transferencia quedó `COMPENSADA` en el intento 8, con Elena de vuelta en 1000 y Fer en 500.

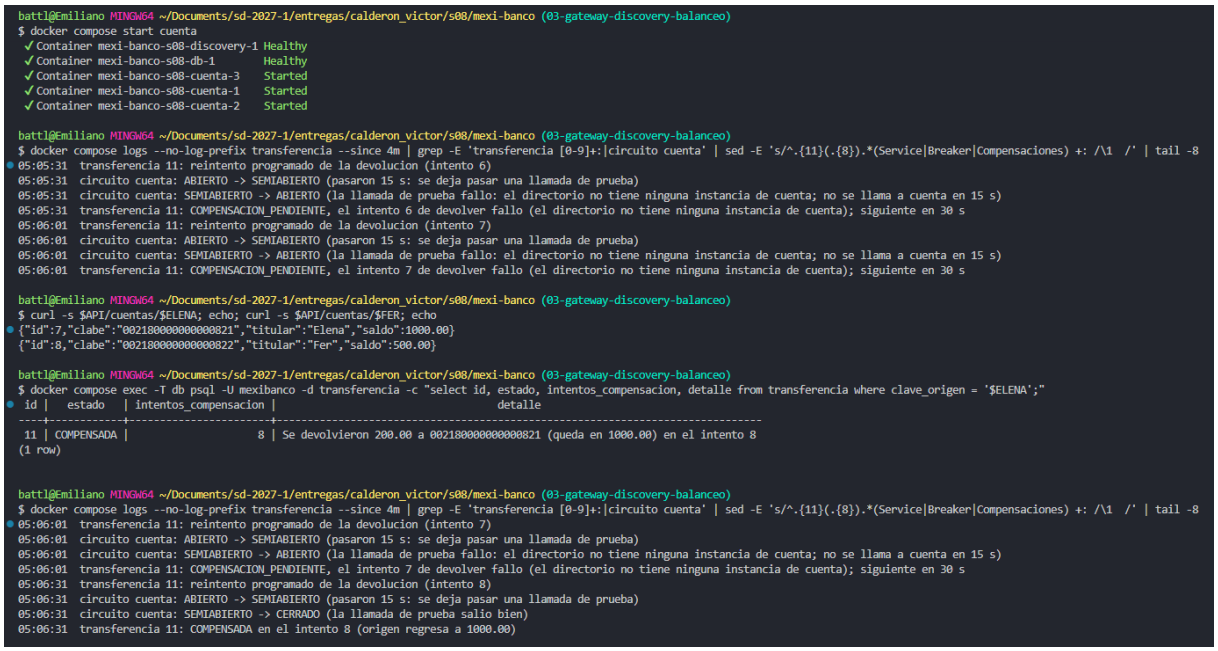


Figura 10: Con `cuenta` de vuelta: el intento 8 devuelve el dinero, el circuito se cierra y la transferencia queda `COMPENSADA`.

¿Cómo la terminaría?

En mi caso ya no se queda pendiente para siempre, porque lo que hice fue justo un reintento programado: `ReintentoDeCompensaciones` revisa la base cada segundo y vuelve a intentar las devoluciones a las que ya les toca, con una espera que va creciendo (2, 4, 8, 16 y hasta 30 segundos) para no estar golpeando a `cuenta` mientras está caída. Como el estado vive en la base y no en la memoria del proceso, si `transferencia` se reiniciara a la mitad, al volver a arrancar retomaría las pendientes. Lo que le agregaría para un banco de verdad es un límite: si después de cierto tiempo (por ejemplo, una hora) la devolución sigue sin salir, que se dé aviso a una persona para revisarla a mano, porque un reintento infinito tampoco resuelve todo.

¿Por qué la compensación debe ser idempotente?

Porque la misma devolución se puede ejecutar más de una vez: la intenta la saga, luego el reintento programado, y también la podría intentar un `transferencia` recién reiniciado o una segunda copia del servicio. Si devolver no fuera idempotente, cada repetición le abonaría otros 200 a Elena, es decir, el banco estaría creando dinero de la nada (la regla de la clase de "reintentar sin idempotencia cobra dos veces", aquí al revés: devuelve dos veces). Mi versión lo cuida del lado de `transferencia` de tres formas: `compensar()` solo actúa si la transferencia está `CARGADA` o `COMPENSACION_PENDIENTE` (una vez `COMPENSADA`, llamarla otra vez no hace nada, y eso lo comprueba una de las pruebas); antes de llamar a `cuenta` aparta la fila en la base y, con el `@Version`, si otro proceso ya la había apartado el guardado truena y ese proceso no llama a `cuenta`; y en cuanto `cuenta` confirma la devolución, lo primero que hace es guardar `COMPENSADA`.

Aun así, hay un caso que `transferencia` sola no puede cerrar: si el abono a origen sí llega a `cuenta` y se aplica, pero la respuesta se pierde (por ejemplo, por el timeout de 2 segundos) o `transferencia` se cae justo antes de guardar `COMPENSADA`, entonces `transferencia` cree que falló y lo volvería a intentar. Para cerrarlo, `cuenta` tendría que aceptar una llave de idempotencia en sus cargos y abonos, igual que el SPEI con su `Idempotency-Key`: por ejemplo `transferencia-11-devolucion`, guardada con un `UNIQUE` en la misma transacción local que mueve el saldo, para que una petición repetida con la misma llave regrese el resultado anterior sin volver a aplicarse (el "`s.id()` + paso como llave" que vimos en la clase). No lo cambié porque el lab pide solo `transferencia`, pero sería el siguiente paso.

6. El circuit breaker

Hora: 04:58

Para ver los tres estados, apagué `cuenta` y en la misma línea mandé seis transferencias seguidas, imprimiendo para cada una el código, cuánto tardó y el detalle de la respuesta:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s08/mexi-banco (03-gateway-discovery-balanceo)
$ docker compose stop cuenta; for i in 1 2 3 4 5 6; do curl -s -w '\n%{http_code} %{time_total}\n' -X POST $API/transferencias -H 'Content-Type: application/json' -d "{\"claveOrigen\":\"$ANA\",\"c
read -r codigo tiempo; echo "$(date +%T) intento $i: HTTP $codigo en $tiempo s -> $(jq -r .detail <<< "$cuerpo")"; }; done
[+] stop 3/3
 ✓ Container mexi-banco-s08-cuenta-2 Stopped
 ✓ Container mexi-banco-s08-cuenta-3 Stopped
 ✓ Container mexi-banco-s08-cuenta-1 Stopped
04:58:20 intento 1: HTTP 503 en 1.017556 s -> No se movio dinero: cuenta no contesto (Connect timed out)
04:58:21 intento 2: HTTP 503 en 1.022486 s -> No se movio dinero: cuenta no contesto (Connect timed out)
04:58:22 intento 3: HTTP 503 en 1.022246 s -> No se movio dinero: cuenta no contesto (Connect timed out)
04:58:22 intento 4: HTTP 503 en 0.011717 s -> No se movio dinero: No se llamo a cuenta: el circuito esta ABIERTO (se vuelve a probar en 15 s; ultima falla: cuenta no contesto (Connect timed out))
04:58:22 intento 5: HTTP 503 en 0.011187 s -> No se movio dinero: No se llamo a cuenta: el circuito esta ABIERTO (se vuelve a probar en 15 s; ultima falla: cuenta no contesto (Connect timed out))
04:58:23 intento 6: HTTP 503 en 0.010320 s -> No se movio dinero: No se llamo a cuenta: el circuito esta ABIERTO (se vuelve a probar en 15 s; ultima falla: cuenta no contesto (Connect timed out))
```

Figura 11: Con `cuenta` apagada: las tres primeras tardan un segundo; ya con el circuito abierto, las demás responden en 11 milisegundos.

Las tres primeras tardaron 1.02 segundos cada una y fallaron con `Connect timed out`: el balanceador todavía tenía en su lista las tres copias de `cuenta` y trató de conectarse a contenedores que ya no existían, hasta que lo cortó el timeout de conexión de 1 segundo que le puse a `cuenta`. A la tercera falla el circuito pasó de `CERRADO` a `ABIERTO`, y las otras tres respondieron en unos 11 milisegundos con "No se llamo a cuenta: el circuito esta ABIERTO". Es decir, respondió casi cien veces más rápido y sin tocar la red, que es justo la idea: ya no le sigue insistiendo a un servicio que está caído. El endpoint del circuito lo confirma:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s08/mexi-banco (03-gateway-discovery-balanceo)
$ docker compose exec -T transferencia wget -qO- http://localhost:8080/actuator/circuitbreakers | jq .
{
  "cuenta": {
    "estado": "ABIERTO",
    "fallasSeguidas": 3,
    "umbral": 3,
    "esperaAbiertoSegundos": 15,
    "seVuelveAProbarEnSegundos": 0,
    "ultimaFalla": "cuenta no contesto (Connect timed out)",
    "cambios": [
      "04:58:22  CERRADO -> ABIERTO  (3 fallas seguidas; no se llama a cuenta en 15 s)"
    ]
  }
}
```

Figura 12: `/actuator/circuitbreakers` con el circuito abierto.

Aquí `seVuelveAProbarEnSegundos` ya estaba en 0 porque los 15 segundos habían pasado; el paso a semiabierto no ocurre solo, sino cuando llega la siguiente llamada. Luego encendí `cuenta` y mandé una transferencia apenas 4 segundos después. El circuito pasó a `SEMIABIERTO` y dejó pasar esa llamada de prueba, pero `cuenta` todavía no terminaba de arrancar ni de registrarse en el directorio, por lo que la prueba falló y el circuito se volvió a abrir con la espera desde cero (y la transferencia quedó `RECHAZADA`, sin mover nada):

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s08/mexi-banco (03-gateway-discovery-balanceo)
$ docker compose start cuenta
[+] start 5/5
 ✓ Container mexi-banco-s08-discovery-1 Healthy
 ✓ Container mexi-banco-s08-db-1        Healthy
 ✓ Container mexi-banco-s08-cuenta-3    Started
 ✓ Container mexi-banco-s08-cuenta-1    Started
 ✓ Container mexi-banco-s08-cuenta-2    Started

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s08/mexi-banco (03-gateway-discovery-balanceo)
$ docker compose ps cuenta
NAME                     IMAGE                    COMMAND              SERVICE   CREATED         STATUS                            PORTS
mexi-banco-s08-cuenta-1   mexi-banco-s08-cuenta   "java -jar app.jar"  cuenta    16 minutes ago  Up 4 seconds (health: starting)   8080/tcp
mexi-banco-s08-cuenta-2   mexi-banco-s08-cuenta   "java -jar app.jar"  cuenta    16 minutes ago  Up 4 seconds (health: starting)   8080/tcp
mexi-banco-s08-cuenta-3   mexi-banco-s08-cuenta   "java -jar app.jar"  cuenta    16 minutes ago  Up 4 seconds (health: starting)   8080/tcp

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s08/mexi-banco (03-gateway-discovery-balanceo)
$ curl -s -X POST $API/transferencias -H 'Content-Type: application/json' -d "{\"claveOrigen\":\"$ANA\",\"claveDestino\":\"$BETO\",\"monto\":10}" | jq -c '
{"transferencia":null,"estado":"RECHAZADA","detalle":null}

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s08/mexi-banco (03-gateway-discovery-balanceo)
$ docker compose exec -T transferencia wget -qO- http://localhost:8080/actuator/circuitbreakers | jq .
{
  "cuenta": {
    "estado": "ABIERTO",
    "fallasSeguidas": 3,
    "umbral": 3,
    "esperaAbiertoSegundos": 15,
    "seVuelveAProbarEnSegundos": 10,
    "ultimaFalla": "el directorio no tiene ninguna instancia de cuenta",
    "cambios": [
      "04:58:22  CERRADO -> ABIERTO  (3 fallas seguidas; no se llama a cuenta en 15 s)",
      "04:59:25  ABIERTO -> SEMIABIERTO  (pasaron 15 s: se deja pasar una llamada de prueba)",
      "04:59:25  SEMIABIERTO -> ABIERTO  (la llamada de prueba fallo: el directorio no tiene ninguna instancia de cuenta; no se llama a cuenta en 15 s)"
    ]
  }
}
```

Figura 13: La primera prueba falla porque `cuenta` sigue arrancando: `SEMIABIERTO` → `ABIERTO`.

Ya con las tres copias sanas, la siguiente transferencia fue otra vez la llamada de prueba, ahora sí salió bien y el circuito se cerró, con la transferencia `COMPLETADA`. En la lista de cambios del endpoint queda el ciclo completo con la hora de cada uno:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s08/mexi-banco (03-gateway-discovery-balanceo)
$ curl -s -X POST $API/transferencias -H 'Content-Type: application/json' -d "{\"claveOrigen\":\"$ANA\",\"claveDestino\":\"$BETO\",\"monto\":10}" | jq -c
{"transferencia":10,"estado":"COMPLETADA","detalle":"Cargo, abono y sus dos asientos"}

battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s08/mexi-banco (03-gateway-discovery-balanceo)
$ docker compose exec -T transferencia wget -qO- http://localhost:8080/actuator/circuitbreakers | jq .
{
  "cuenta": {
    "estado": "CERRADO",
    "fallasSeguidas": 0,
    "umbral": 3,
    "esperaAbiertoSegundos": 15,
    "ultimaFalla": "el directorio no tiene ninguna instancia de cuenta",
    "cambios": [
      "04:58:22  CERRADO -> ABIERTO  (3 fallas seguidas; no se llama a cuenta en 15 s)",
      "04:59:25  ABIERTO -> SEMIABIERTO  (pasaron 15 s: se deja pasar una llamada de prueba)",
      "04:59:25  SEMIABIERTO -> ABIERTO  (la llamada de prueba fallo: el directorio no tiene ninguna instancia de cuenta; no se llama a cuenta en 15 s)",
      "05:02:10  ABIERTO -> SEMIABIERTO  (pasaron 15 s: se deja pasar una llamada de prueba)",
      "05:02:10  SEMIABIERTO -> CERRADO  (la llamada de prueba salio bien)"
    ]
  }
}
```

Figura 14: La prueba sale bien: `SEMIABIERTO` → `CERRADO`, y la lista de todos los cambios.

Y lo mismo se ve en los logs de `transferencia`, donde además están las tres fallas que fue contando antes de abrirse:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s08/mexi-banco (03-gateway-discovery-balanceo)
$ docker compose logs --no-log-prefix transferencia --since 10m | grep 'circuito cuenta' | sed -E 's/^.{11}(.{8}).*CircuitBreaker +: /\1  /'
04:58:20  circuito cuenta: falla 1 de 3 (cuenta no contesto (Connect timed out))
04:58:21  circuito cuenta: falla 2 de 3 (cuenta no contesto (Connect timed out))
04:58:22  circuito cuenta: falla 3 de 3 (cuenta no contesto (Connect timed out))
04:58:22  circuito cuenta: CERRADO -> ABIERTO (3 fallas seguidas; no se llama a cuenta en 15 s)
04:59:25  circuito cuenta: ABIERTO -> SEMIABIERTO (pasaron 15 s: se deja pasar una llamada de prueba)
04:59:25  circuito cuenta: SEMIABIERTO -> ABIERTO (la llamada de prueba fallo: el directorio no tiene ninguna instancia de cuenta; no se llama a cuenta en 15 s)
05:02:10  circuito cuenta: ABIERTO -> SEMIABIERTO (pasaron 15 s: se deja pasar una llamada de prueba)
05:02:10  circuito cuenta: SEMIABIERTO -> CERRADO (la llamada de prueba salio bien)
```

Figura 15: Los cambios de estado del circuito en los logs.

Algo que me gustó es que las transferencias que el circuito rechazó tampoco se perdieron: en la base quedaron como `RECHAZADA` (de la 3 a la 9), con el motivo de cada una.

7. Lo que la saga no te da

Hora: 05:04

En la clase vimos que una saga da atomicidad (eventual), consistencia y durabilidad, pero no da aislamiento, y en la prueba de la sección 5 se ve muy claro. Tres segundos después de lanzar la transferencia de Elena, mientras estaba en la pausa, consulté su cuenta como lo haría cualquier otro cliente:

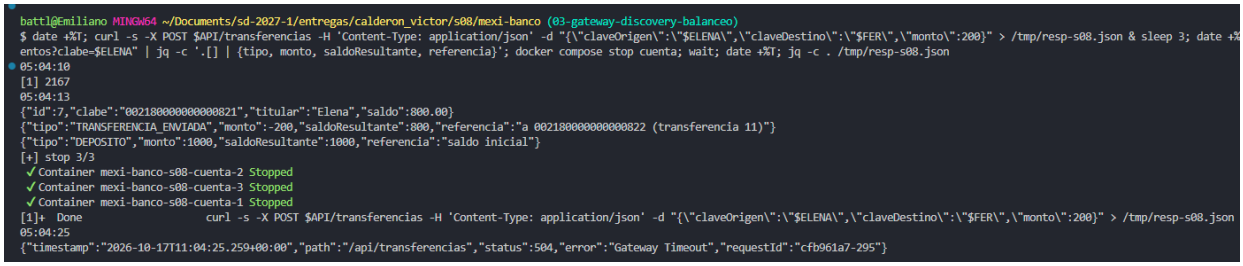


Figura 16: A las 05:04:13 (a mitad de la saga) cualquiera ve a Elena con 800 y el cargo de la transferencia 11 en su estado de cuenta.

Es decir, cualquiera que consultara el saldo de Elena entre el cargo y la compensación veía 800 y un cargo de 200 en su estado de cuenta, como si la transferencia ya hubiera pasado, cuando en realidad esa transferencia nunca se completó: terminó `COMPENSADA` a las 05:06:31 y Elena regresó a 1000. Por lo que durante 2 minutos y 21 segundos el sistema mostró un estado intermedio como si fuera definitivo, ya que cada paso de la saga es una transacción local que se confirma y se vuelve visible en ese momento, y no hay ningún candado que cubra la saga completa. Si Elena hubiera querido pagar algo de 900 en ese rato, se lo habrían rechazado por saldo insuficiente aunque al final tenía 1000, y un reporte que sumara los saldos de todas las cuentas en ese momento estaría "perdiendo" 200 (el mismo caso de *read skew* que vimos en la clase, pero ahora entre servicios). Lo que se puede hacer, según lo que vimos, es marcar lo que todavía no es definitivo: mi `transferencia` sí sabe que esa operación está `CARGADA` o `COMPENSACION_PENDIENTE`, pero `cuenta` no, por lo que su saldo no distingue entre dinero que ya se fue y dinero que está "en tránsito"; una mejora sería que `cuenta` manejara un saldo retenido aparte del disponible, y que el estado de cuenta marcara los movimientos de transferencias que todavía no terminan.

Entrega

En la carpeta `codigo` van dos cosas: `transferencia`, que es la carpeta completa del servicio sin `target/`, y el `docker-compose.yml` con mis cambios. Para levantarlo hay que copiar las dos sobre la rama base del lab (`03-gateway-discovery-balanceo`), y para las pruebas de esta bitácora usé:

```
docker compose up --build -d
FALLA_ABONO_CLABE=002180000000000812 docker compose up -d transferencia   # la falla provocada
PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia      # la pausa de la Parte 0
docker compose exec -T transferencia wget -qO- http://localhost:8080/actuator/circuitbreakers
```